

ARCore Depth Hazard Detection

白杖では捉えにくい高所障害物の距離と下端高さを、対応するAndroidスマートフォンで推定する
研究用アプリ

135

評価試行

27条件 × 各5回

88.97%

高さ近似率

検知できた試行の平均

4.181 m

平均初検知距離

評価実験における平均

課題

吊り下げ看板や木の枝など、胸部・頭部付近の障害物は白杖の探索範囲に入りにくく、衝突前の把握が難しい。

提案

専用LiDARを必須とせず、ARCore DepthとRGB画像処理を組み合わせることで障害物下端の高さを推定する。

開発形態

個人開発・2025年度卒業研究

担当範囲

課題設定、手法設計、Android実装、評価実験、統計分析、論文執筆

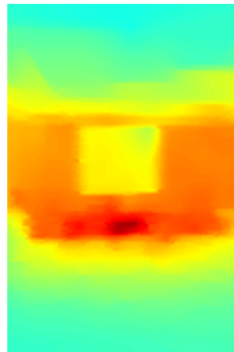
実験端末

Google Pixel 7a

提案手法 — 深度とRGBの弱点を補完する



RGB画像



深度画像



二値化画像



エッジ画像

1

深度取得

ARCoreから16 bit深度画像とカメラ内部パラメータを取得

2

動的二値化

画面上部20%の深度中央値を、フレームごとの背景深度に設定

3

領域の分離

検出範囲を制限し、収縮処理で床・壁との微細な連結を切り離す

4

下端補正

RGB画像のCannyエッジで、低解像度深度画像の境界を補正

5

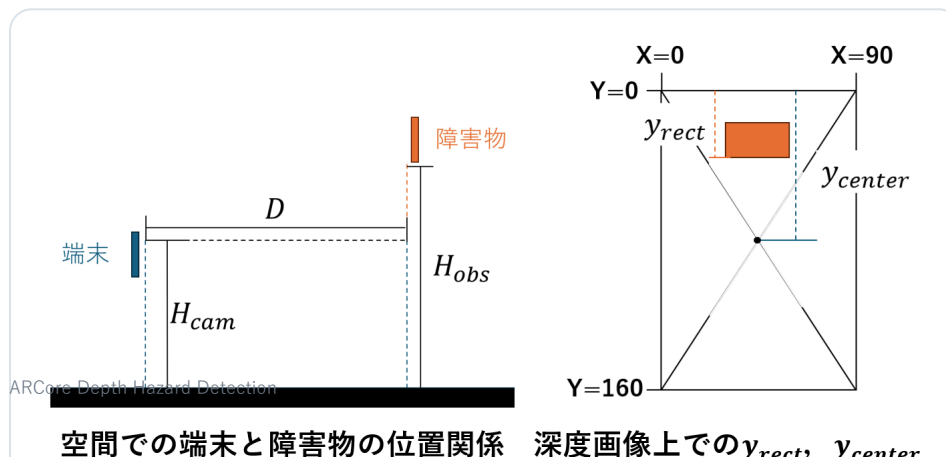
高さ推定

深度中央値とピンホールカメラモデルから距離・下端高さを算出

6

評価可能な記録

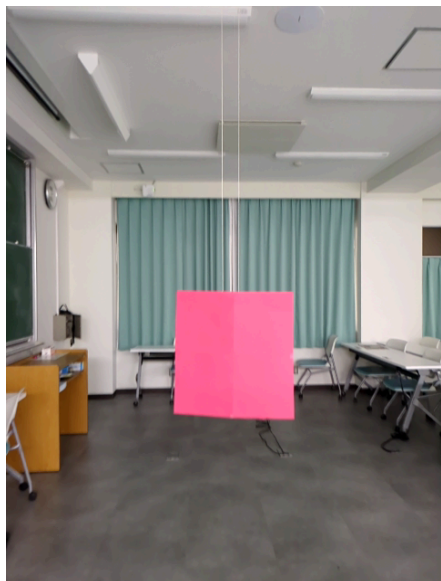
推定値、距離、マスク、深度画像をJSON・PNGとして保存



設計上の工夫

- 平均値ではなく中央値を使い、深度の飛び値に耐える
- 焦点距離を端末から取得し、端末固有の内部パラメータへ対応
- 画像処理を10フレームごとに実行し、リアルタイム描画との負荷を分離

評価 — 精度と信頼性を分けて検証



実験条件

- 色：黒・赤・白
- 一辺：500 / 300 / 100 mm
- 下端高さ：1.0 / 1.4 / 1.8 m
- 27条件 × 5回 = 135 試行
- 開始距離5.0 m、カメラ高1.5 m

88.97%

高さ近似率

検知できた試行の平均

31.62%

有効データ率

高さ推定に成功したフレームの割合

p = 0.00001

大きさの有意差

F(2,132) = 12.57

障害物サイズ別の平均近似率

一辺	500 mm	300 mm	100 mm
近似率	91.56%	87.48%	58.21%

結果の解釈

高さを推定できた場合の精度は高い一方、有効データ率は31.62%にとどまった。小さい障害物ほど深度画像上の領域が不足し、検知が不安定になることを定量的に確認した。

成果と次の改善

身につけた・実践したこと

- 社会課題から検証可能な研究課題へ落とし込む
- ARCore、OpenCV、カメラ幾何を統合したAndroid実装
- 比較可能な条件設計と135試行のデータ収集
- 成功例だけでなく、有効データ率と失敗条件を分析
- 一元配置分散分析による要因の定量評価
- GitHub Actionsによる再現可能なビルド環境の整備

Kotlin

Android

ARCore

OpenCV

OpenGL ES

Computer Vision

Accessibility

明らかになった制約

- 小さい物体や床面に近い物体の検知が不安定
- 照明、反射、端末姿勢、色の組み合わせの影響を受ける
- ユーザーへの音声・振動通知は未実装
- 実環境・複数端末・当事者を含む評価は未実施

今後取り組むこと

- 環境に応じた閾値と検出領域の適応化
- 小物体検知と有効データ率の改善
- 画像処理ロジックのモジュール化とテスト
- 音声・振動を用いた安全な通知設計

この研究では「動いた」で終わらず、精度・検知安定性・失敗条件を分けて評価しました。技術を社会課題へ適用し、実験結果から次の改善点を具体化するまでを一貫して経験したプロジェクトです。

SOURCE CODE / README / BUILD STATUS

github.com/Jr34374/arcore-depth-hazard-detection

研究用プロトタイプであり、実際の歩行時の安全装置として単独で使用できるものではありません。